Combinatorics & Probability

Counting, inclusion-exclusion, and expected value

Learning goals

- Count with the product rule, permutations, combinations, multinomials
- Compute nCr mod p for many queries (factorials + inverse factorials)
- Recognize **stars and bars** and **Catalan numbers**
- Apply **inclusion-exclusion** (derangements, divisibility, "all different")
- Use **linearity of expectation** and indicator variables
- Write **probability / expected-value DPs**, including answers mod p

Toolkit recap (Week 11)

```cpp
const int MOD = 1'000'000'007, MAXN = 2'000'001;
long long fact[MAXN], inv_fact[MAXN];
long long power(long long b, long long e) {
    long long r = 1; b %= MOD;
    for (; e; e >>= 1, b = b * b % MOD) if (e & 1) r = r * b % MOD;
    return r;
}
void init() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    inv_fact[MAXN - 1] = power(fact[MAXN - 1], MOD - 2);
    for (int i = MAXN - 1; i > 0; i--) inv_fact[i - 1] = inv_fact[i] * i % MOD;
}
long long C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
}
```

O(MAXN) precomputation, O(1) per query.

The basic counts

Objects	Count
sequences of length k over n symbols	n^k
orderings of n distinct items	$n!$
ordered choice of k from n	$n!/(n-k)!$
subsets of size k	$C(n,k) = n!/(k!(n-k)!)$
arrangements of a multiset (counts $c_1..c_m$)	$n!/(c_1!\cdot\ldots\cdot c_m!)$

Example (CSES 1715): permutations of "aabac" = $5!/(3!\cdot1!\cdot1!) = 20$.

Stars and bars

> Distribute **m identical** apples to **n children** (zero allowed).

Write m stars and n−1 bars in a row; bars split the stars into n groups:

```
**|*||***    →   (2, 1, 0, 3)     n = 4, m = 6
```

Answer: choose positions of the bars among m + n − 1 slots → **C(m+n−1, n−1)**.

- At least one each: give one first → C(m−1, n−1)
- Upper bounds per child → combine with inclusion-exclusion

Catalan numbers

$C_n = C(2n, n) / (n+1) : 1, 1, 2, 5, 14, 42, 132, \ldots$

Counts: balanced bracket sequences with n pairs, binary trees with n nodes, monotone lattice paths not crossing the diagonal, triangulations of an (n+2)-gon.

Reflection argument: bad paths (touching $y = -1$) $\leftrightarrow$ all paths ending at a reflected endpoint.

```
Cₙ = C(2n, n) − C(2n, n+1)
```

CSES 2064 Bracket Sequences I: n odd → 0, else C_{n/2}.

Inclusion-exclusion

For sets $A_1, \ldots, A_k$:

$$|A_1 \cup \ldots \cup A_k| = \Sigma|A_i| - \Sigma|A_i \cap A_j| + \Sigma|A_i \cap A_j \cap A_l| - \ldots$$
$$= \Sigma \text{ over non-empty } S \subseteq \{1..k\} \text{ of } (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right|$$

Recipe:

1. Define "bad" properties A_i
2. Make sure **intersections** are easy to count
3. Sum over subsets (2^k terms — bitmasks!) or over sizes when symmetric

Example: Prime Multiples (CSES 2185)

> How many of 1..n ($n \le 10^{18}$) are divisible by at least one of $k \le 20$ primes?

Numbers divisible by all primes in S: $\lfloor n / \prod S \rfloor$.

```
long long ans = 0;
for (int mask = 1; mask < (1 << k); mask++) {
    long long prod = 1; bool over = false;
    for (int i = 0; i < k && !over; i++) if (mask >> i & 1) {
        if (prod > n / a[i]) over = true;        // product exceeds n
        else prod *= a[i];
    }
    if (over) continue;                          // contributes 0
    ans += (__builtin_popcount(mask) & 1 ? 1 : -1) * (n / prod);
}
```

$O(2^k \cdot k)$. Watch the **overflow check**!

Example: derangements

> Permutations of n with **no fixed point** (CSES 1717 Christmas Party).

A_i = "i is fixed". Any j fixed points: (n−j)! permutations, C(n,j) ways to choose them.

```
D(n) = Σ_{j=0..n} (−1)ʲ C(n,j) (n−j)! = n! Σ (−1)ʲ / j!
```

Recurrence: **D(n) = (n−1)(D(n−1) + D(n−2))**, D(0)=1, D(1)=0.

D: 1, 0, 1, 2, 9, 44, 265, … and D(n)/n! → 1/e.

Symmetric inclusion-exclusion

If |intersection of any j sets| depends only on j, sum over j instead of subsets:

> **ABC 172 E (NEQ):** count pairs (A, B) of length-N sequences over 1..M, each with distinct elements, with $A_i \neq B_i$ for all i.

- Fix A: P(M, N) ways, where P(M,N) = M!/(M−N)!
- B with ≥ the j chosen positions equal: P(M−j, N−j)

```
answer = P(M,N) · Σ_{j=0..N} (−1)ʲ C(N,j) P(M−j, N−j)
```

O(N) with factorials.

Probability basics

- Uniform: $P(E) = |E| / |\Omega|$ — probability is counting divided by counting
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- Independent: $P(A \cap B) = P(A) \cdot P(B)$
- Conditional: $P(A \mid B) = P(A \cap B) / P(B)$

Two outputs you will see:

1. **Floating point** with 1e−6 tolerance → `printf("%.10f")`
2. **Modular:** $P = a/b$ → print $a \cdot b^{-1} \bmod p$

Expected value

$E[X] = \Sigma\, x \cdot P(X = x)$

Linearity of expectation: $E[X + Y] = E[X] + E[Y]$ — **always**, even if X, Y are dependent!

Indicator trick: write $X = \Sigma\, I_i$ where $I_i \in \{0,1\}$; then $E[X] = \Sigma\, P(\text{event } i)$.

- Fixed points of a random permutation: $\Sigma\, P(\pi(i) = i) = n \cdot (1/n) = \mathbf{1}$
- Inversions of a random permutation: $C(n,2) \cdot ½ = \mathbf{n(n-1)/4}$

Example: Game on Tree (CF 280C)

> Repeatedly pick a uniformly random remaining node and delete its whole subtree. Expected number of steps?

- $I_v = 1$ if v is **picked** (not deleted as part of another's subtree)
- v is picked iff it is chosen **before all its ancestors**
- Among v and its d−1 ancestors, each is equally likely to be first → P = 1/d

```
E[steps] = Σᵥ 1 / depth(v)     (root has depth 1)
```

A hard-looking process becomes a one-line DFS.

Geometric distribution & coupon collector

- Trial succeeds with probability p → expected trials **1/p**
- Coupon collector: n coupon types, uniform draws
  - having i types, next new one has p = (n−i)/n
  - E = $\sum_i n/(n-i) = \mathbf{n \cdot H_n \approx n \ln n}$

Linearity again: total time = sum of waiting times of each phase.

Expected maximum (CSES 1727 / CF 453A)

> n children each get uniform 1..k candies. Expected maximum?

Compute the **CDF** first: P(max ≤ x) = (x/k)ⁿ.

```
P(max = x) = (x/k)ⁿ − ((x−1)/k)ⁿ
E[max]     = Σ_{x=1..k} x · [(x/k)ⁿ − ((x−1)/k)ⁿ]
```

Alternatively E[max] = Σ_{x≥1} P(max ≥ x) — the tail-sum formula.

Probability DP

> CSES 1725: throw a die n times; probability the sum is in [a, b]?

```
vector<double> dp(6 * n + 1, 0.0); dp[0] = 1;
for (int t = 0; t < n; t++) {
    vector<double> nd(6 * n + 1, 0.0);
    for (int s = 0; s <= 6 * t; s++)
        for (int f = 1; f <= 6; f++) nd[s + f] += dp[s] / 6;
    dp = nd;
}
```

State = "what we know so far", transitions weighted by probabilities. Forward pushing of **probability mass**.

Expected-value DP

Usually computed **backwards from the end**: E[state] = cost + Σ P(next) · E[next].

Self-loops: $E = 1 + p \cdot E + (1-p) \cdot E' \Longrightarrow E = (1 + (1-p) \cdot E') / (1 - p)$

> **CF 1265E Beautiful Mirrors:** mirror i says "beautiful" w.p. p_i; on success go on, on failure restart at mirror 1. Expected days?

Let $P_j = p_1 \ldots p_j$. Unrolling gives

```
E = (1 + P₁ + P₂ + … + P_{n−1}) / Pₙ
```

computed mod 998244353 with modular inverses.

Modular probabilities

"Print the answer as $P \cdot Q^{-1} \bmod 998244353$":

- Represent probability ½ as `inv(2)`, a percentage p as `p · inv(100)`
- Add/multiply as usual mod p; divide by multiplying by `power(x, MOD-2)`
- **Never** convert to double in between
- Valid because the true answer is a rational with denominator coprime to MOD

Complexity cheat sheet

Task	Cost
factorials up to N	$O(N)$ once, then $O(1)$ per nCr
Pascal triangle (any modulus)	$O(N^2)$
inclusion-exclusion over k sets	$O(2^k \cdot k)$
symmetric inclusion-exclusion	$O(k)$
probability DP	#states × #transitions
expected value via linearity	usually $O(n)$

Common pitfalls

- nCr with k < 0 or k > n must return **0**
- Factorial table too small (stars and bars needs n + m − 1!)
- Overflow when multiplying primes in inclusion-exclusion
- Sign errors: odd-size subsets **add**, even-size **subtract** (for the union)
- Forgetting negative mod: `(a - b + MOD) % MOD`
- Assuming independence where there is none — linearity doesn't need it, products do
- Printing doubles with too few digits

Practice problems

- **CSES 1715** Creating Strings II · **CSES 1716** Distributing Apples
- **CSES 2064** Bracket Sequences I · **CSES 1717** Christmas Party
- **CSES 2185** Prime Multiples · **AtCoder ABC 172 E** NEQ
- **CSES 1725** Dice Probability · **CSES 1727** Candy Lottery
- **CSES 1728** Inversion Probability · **CF 453A** Little Pony and Expected Maximum
- **CF 280C** Game on Tree · **CF 1265E** Beautiful Mirrors
- **AtCoder DP I** Coins · **AtCoder DP J** Sushi

Summary

- Counting = product rule + nCr + multinomials; stars and bars; Catalan
- **Inclusion-exclusion** turns "at least one / none" into easy intersections
- **Linearity of expectation** + indicators solve many "expected" problems in O(n)
- Probability & expected-value **DPs**: forward mass, backward expectation
- Modular answers: rationals as $a \cdot b^{-1} \bmod p$

Next week: computational geometry I.